A Predictive Analysis of Amazon Best Sellers

Dulce Ximena Cid Sanabria, Marco Alejandro Ortiz

2026-05-07

Introduction and Problem Statement

The objective is to predict the binary indicator `isBestSeller` from product-level attributes available at listing time (price, star rating, review volume, and category). The outcome is meaningful: Best Seller status drives placement, click-through, and conversion on the marketplace, so a model that ranks listings by their probability of attaining that status is directly useful for assortment, pricing, and promotional decisions. The class is rare (0.67% of the cleaned sample), which fixes the inferential goal as ranking and probability calibration rather than discrete classification at a fixed threshold.

The Amazon catalog provides 1.4 million independent listings spanning 248 categories, which permits rigorous non-linear inference: smoothing splines on continuous reputation variables, L_1 regularization on a one-hot encoding of categories, a multilevel logistic that exploits the natural nesting of small categories within coarse parent groups, a kernel SVM testing for unstructured non-linear interaction, and an evaluation under PR-AUC.

Data and Pre-processing

The Amazon Products data set contains 1,426,337 rows and 11 columns, paired with a small categories lookup file. Columns `imgUrl`, `productURL`, and `listPrice` carry no predictive content for sales success and are dropped. Observations with `price = 0` or `stars = 0` correspond to unlisted or unrated entries rather than valid market offerings, and are removed. The two files are joined on `category_id` to attach category labels. Because both `price` and `reviews` exhibit heavy right skew, the analysis works on `log_price = log(price)` and `log_reviews = log1p(reviews)`. The latter handles the dense mass at zero reviews without discarding observations.

```
products   <- read_csv("amazon_products.csv", show_col_types = FALSE)
categories <- read_csv("amazon_categories.csv", show_col_types = FALSE)

products <- products %>% select(-imgUrl, -productURL, -listPrice)

n_raw <- nrow(products)

products <- products %>%
  mutate(isBestSeller = as.integer(isBestSeller %in% c("True", TRUE, 1))) %>%
  filter(price > 0, stars > 0)

n_filtered <- nrow(products)

products <- products %>%
  left_join(categories, by = c("category_id" = "id")) %>%
```

```r
  mutate(
    log_price     = log(price),
    log_reviews   = log1p(reviews),
    category_name = factor(category_name)
  )

# Build a coarse "big_category" grouping for the multilevel model and for use
# in the GAM and SVM. Order matters: earlier matches take precedence.
classify_big <- function(x) {
  x <- as.character(x)
  dplyr::case_when(
    grepl("Smart Home", x)
    grepl("Wearable|Virtual Reality", x)
    grepl("Nintendo|PlayStation|Xbox|Wii|PC Games|Mac Games|Video Game|Sony PSP|Online Video Game", x)
    grepl("Computer|Phone|Camera|Audio|TV|Television|Headphone|Earbud|Tablet|Laptop|eBook|GPS|Office El
    grepl("Luggage|Suitcase|Backpack|Travel|Messenger Bag|Garment Bag|Rain Umbrella", x)
    grepl("Clothing|Shoes|Jewelry|Watches|Handbag|Accessor|School Uniform|Sunglass", x)
    grepl("Beauty|Makeup|Hair Care|Skin Care|Personal Care|Perfume|Fragrance|Oral Care|Shaving|Foot, Ha
    grepl("Health|Medical|Vitamin|Diet|Sports Nutrition|Wellness|Vision Products|Sexual Wellness", x)
    grepl("Toy|Doll|Puzzle|Stuffed Animal|Kids' Play|Building Toy|Learning|Finger Toy|Novelty Toy|Puppe
    grepl("Kitchen|Bath Product|Bedding|Furniture|Décor|Decor|Lighting|Light Bulb|Vacuum|Ironing|Heatin
    grepl("Cleaning|Household", x)
    grepl("Tool|Hardware|Building Supplies|Fastener|Cutting|Welding|Power Transmission|Hydraulics|Pneuma
    grepl("Industrial|Lab|Scientific|Abrasive|Additive Manufacturing|Material Handling|Packaging|Retail
    grepl("Automotive|Car Care|Motorcycle|RV |Vehicle", x)
    grepl("Sports|Outdoor|Fitness", x)
    grepl("Pet|Dog|Cat |Bird|Fish|Reptile|Horse|Aquatic|Small Animal", x)
    grepl("Baby|Infant|Toddler|Child Safety|Nursery|Pregnancy|Toilet Training|Maternity", x)
    grepl("Arts|Crafts|Sewing|Knitting|Beading|Scrapbooking|Painting, Drawing|Fabric|Needlework|Printma
    grepl("Office|Stationery|Gift Wrapping|Party|Seasonal|Gift Card", x)
    TRUE
  )
}

products <- products %>%
  mutate(big_category = factor(classify_big(category_name)))

prevalence <- mean(products$isBestSeller)

n_missing <- sum(is.na(products))

tibble(
  metric = c("Raw rows", "After price>0 & stars>0", "Best Seller prevalence", "Missing values (post-clea
  value  = c(comma(n_raw), comma(n_filtered), percent(prevalence, accuracy = 0.01), comma(n_missing))
) %>% as.data.frame()
```

```
##                       metric     value
## 1                   Raw rows 1,426,337
## 2     After price>0 & stars>0 1,267,186
## 3      Best Seller prevalence     0.67%
## 4 Missing values (post-clean)         0
```

The filter retains 89% of the original rows, and the positive class accounts for about 0.67% of the analytic

sample. The cleaned data set carries no missing values, so no imputation is required. The implications of the class imbalance are addressed in the modeling section.

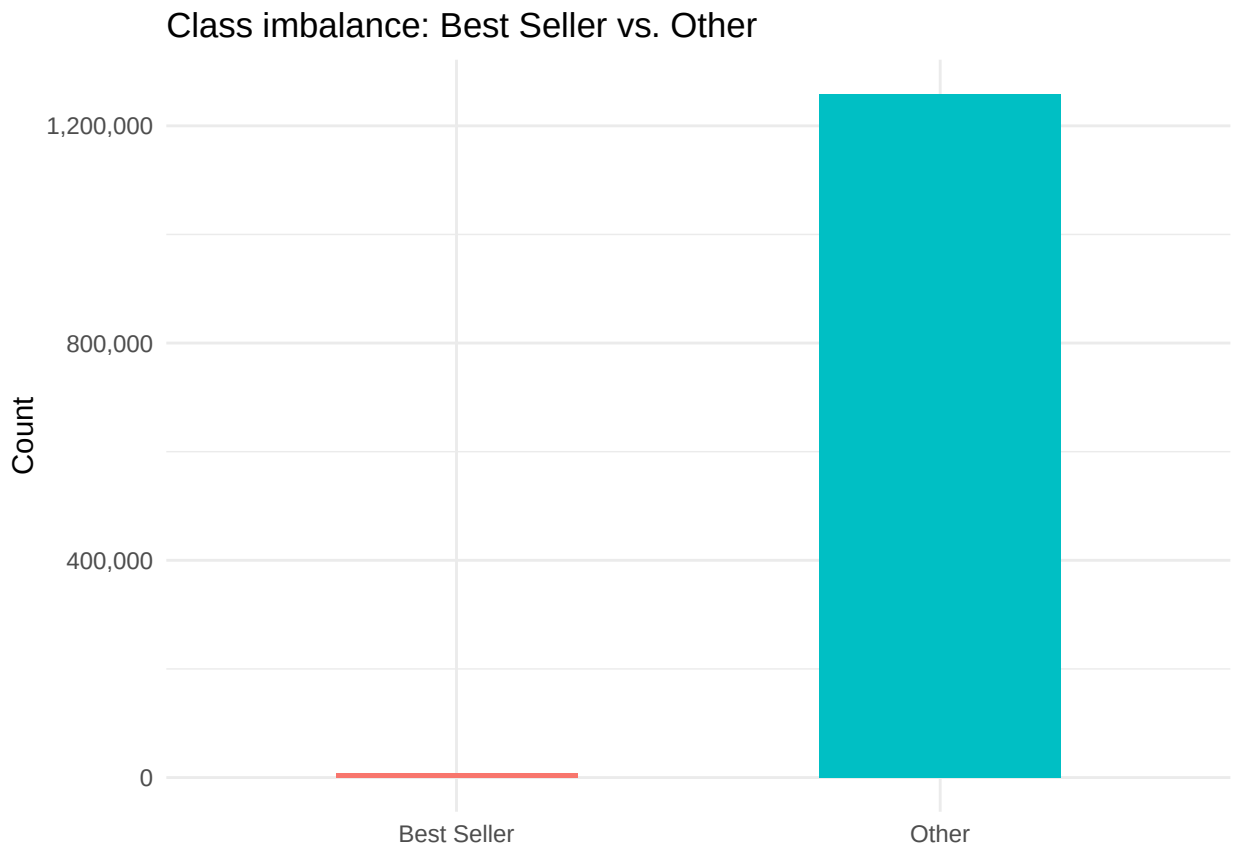
```
descr <- products %>%
  summarise(
    across(c(price, stars, reviews),
           list(Min = min, Median = median, Mean = mean, Max = max, SD = sd),
           .names = "{.col}__{.fn}")
  ) %>%
  pivot_longer(everything(), names_to = c("Variable", "stat"), names_sep = "__") %>%
  pivot_wider(names_from = stat, values_from = value) %>%
  mutate(Variable = recode(Variable,
                           price   = "Price (USD)",
                           stars   = "Stars (1-5)",
                           reviews = "Review count"))

as.data.frame(descr)
```

```
##       Variable  Min Median       Mean    Max           SD
## 1  Price (USD) 0.01  19.99  41.179979  11998  100.3907684
## 2  Stars (1-5) 1.00   4.50   4.404724      5    0.4552581
## 3 Review count 0.00   0.00 197.497947 346563 1826.6491755
```

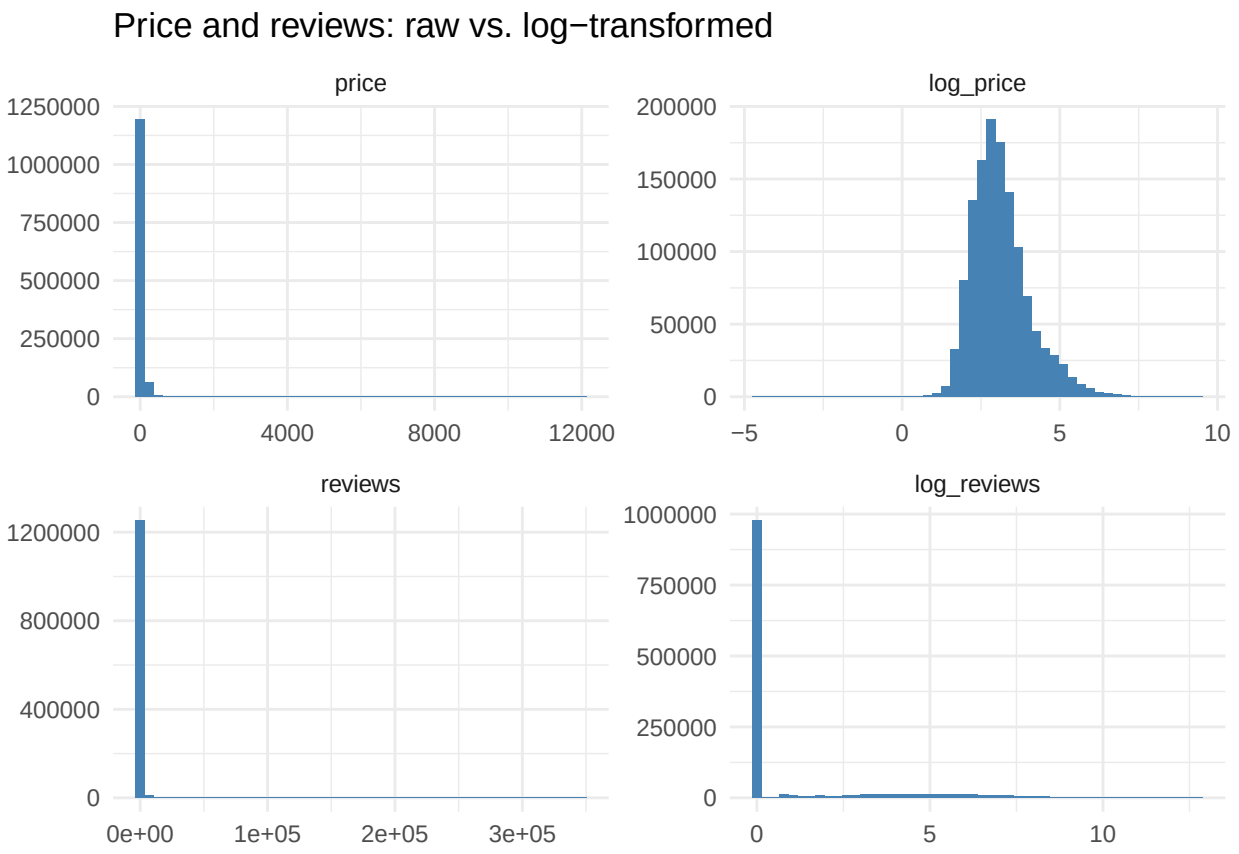
Exploratory Data Analysis

Price and review counts are right-skewed. The panels below establish the empirical regularities that motivate the modeling choices.

```
products %>%
  count(isBestSeller) %>%
  mutate(label = ifelse(isBestSeller == 1, "Best Seller", "Other")) %>%
  ggplot(aes(label, n, fill = label)) +
  geom_col(width = 0.5) +
  scale_y_continuous(labels = comma) +
  labs(title = "Class imbalance: Best Seller vs. Other", x = NULL, y = "Count") +
  theme_minimal() +
  theme(legend.position = "none")
```

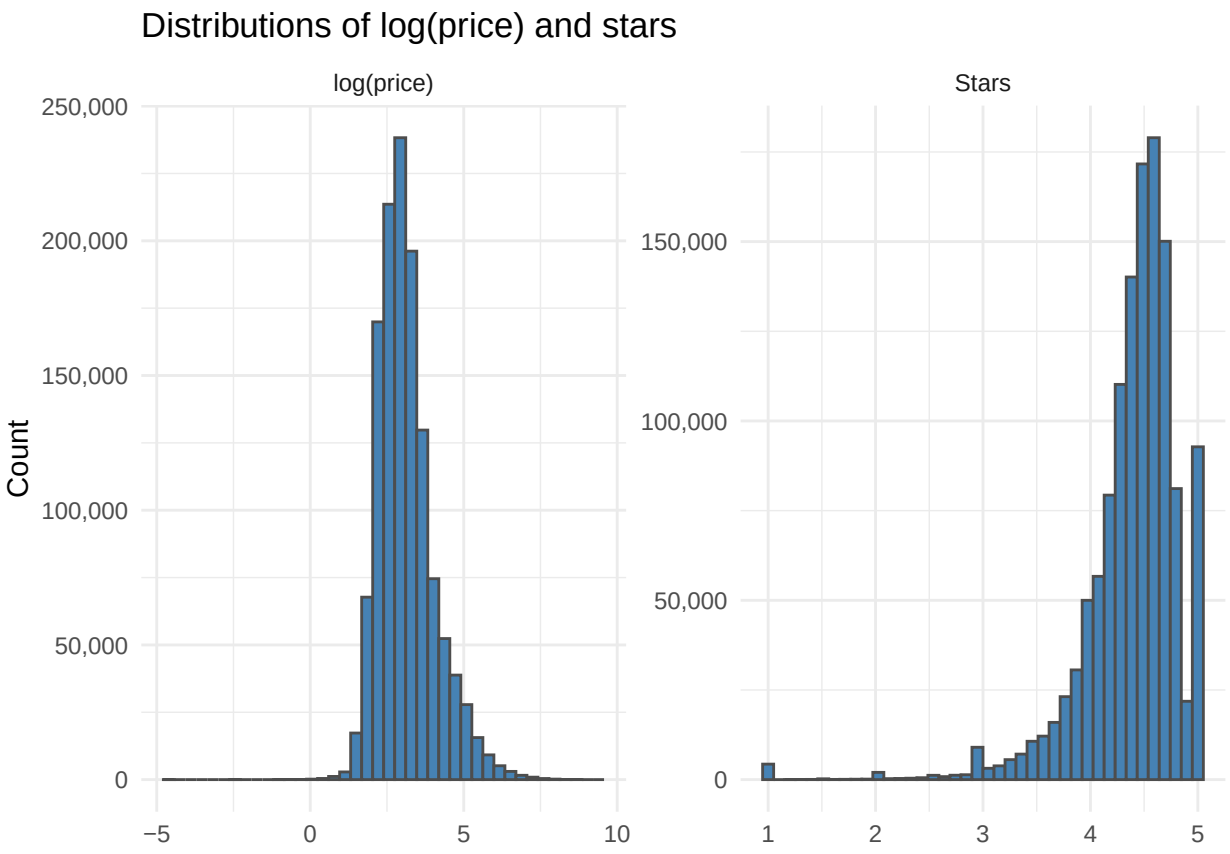



```
products %>%
  select(price, reviews, log_price, log_reviews) %>%
  pivot_longer(everything(), names_to = "var", values_to = "value") %>%
  mutate(var = factor(var, levels = c("price", "log_price", "reviews", "log_reviews"))) %>%
  ggplot(aes(value)) +
  geom_histogram(bins = 50, fill = "steelblue") +
  facet_wrap(~ var, scales = "free", ncol = 2) +
  labs(title = "Price and reviews: raw vs. log-transformed",
       x = NULL, y = NULL) +
  theme_minimal()
```



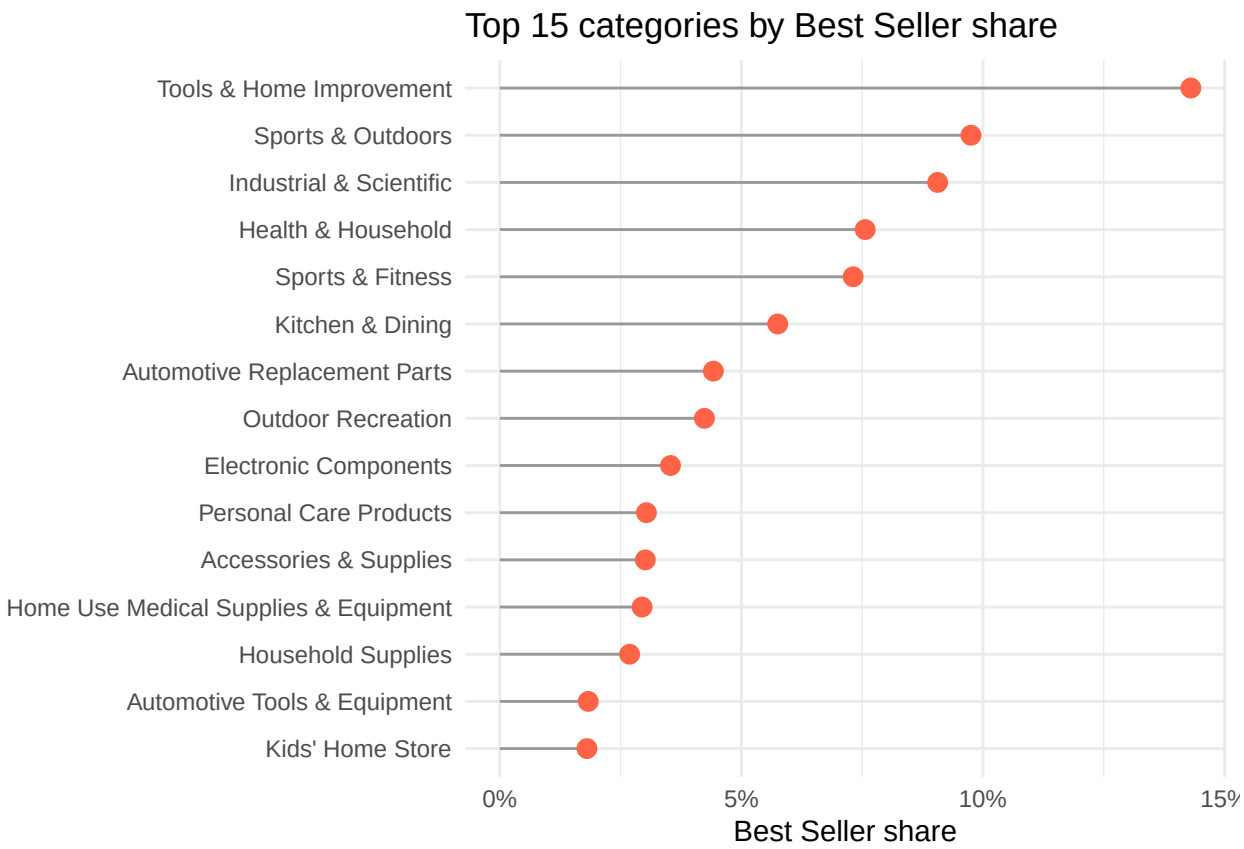
After the log transformation, both predictors become approximately symmetric, with `log_reviews` retaining a dense mass at zero from products with no reviews.

```
products %>%
  select(log_price, stars) %>%
  pivot_longer(everything(), names_to = "var", values_to = "value") %>%
  mutate(var = recode(var,
                      log_price = "log(price)",
                      stars     = "Stars")) %>%
  ggplot(aes(value)) +
  geom_histogram(bins = 40, fill = "steelblue", color = "grey30") +
  facet_wrap(~ var, scales = "free", ncol = 2) +
  scale_y_continuous(labels = comma) +
  labs(title = "Distributions of log(price) and stars",
       x = NULL, y = "Count") +
  theme_minimal()
```



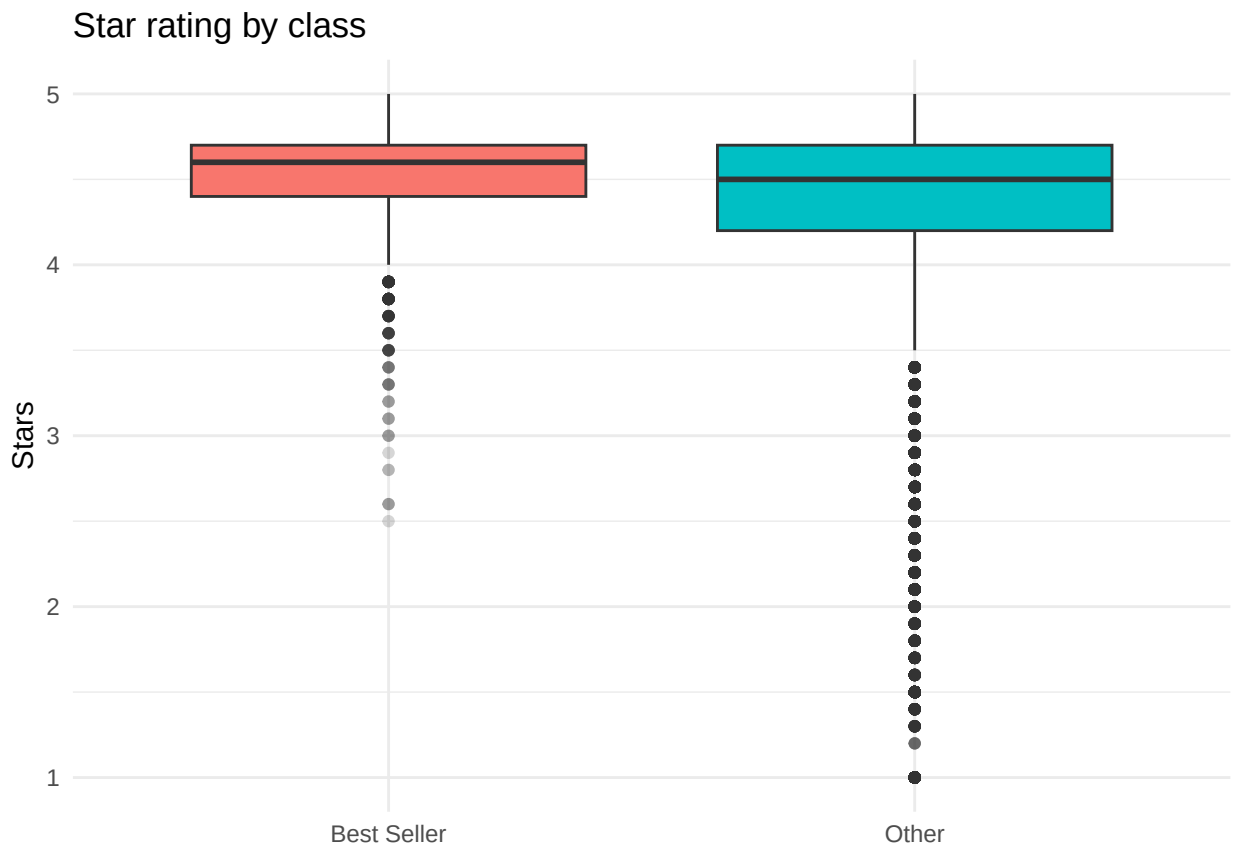
`log(price)` is roughly symmetric around the mid range and is used as a continuous predictor without further transformation. `stars` is highly skewed, with most values near 4.5 and a long lower tail. This shape reflects the well-known selection bias in voluntary rating systems and is the reason `stars` enters the GAM as a linear term rather than a smooth.

```
cat_share <- products %>%
  group_by(category_name) %>%
  summarise(n = n(), share_bs = mean(isBestSeller), .groups = "drop") %>%
  filter(n >= 200) %>%
  slice_max(share_bs, n = 15)

ggplot(cat_share, aes(reorder(category_name, share_bs), share_bs)) +
  geom_segment(aes(xend = category_name, y = 0, yend = share_bs), color = "grey60") +
  geom_point(size = 3, color = "tomato") +
  coord_flip() +
  scale_y_continuous(labels = percent) +
  labs(title = "Top 15 categories by Best Seller share",
       x = NULL, y = "Best Seller share") +
  theme_minimal()
```

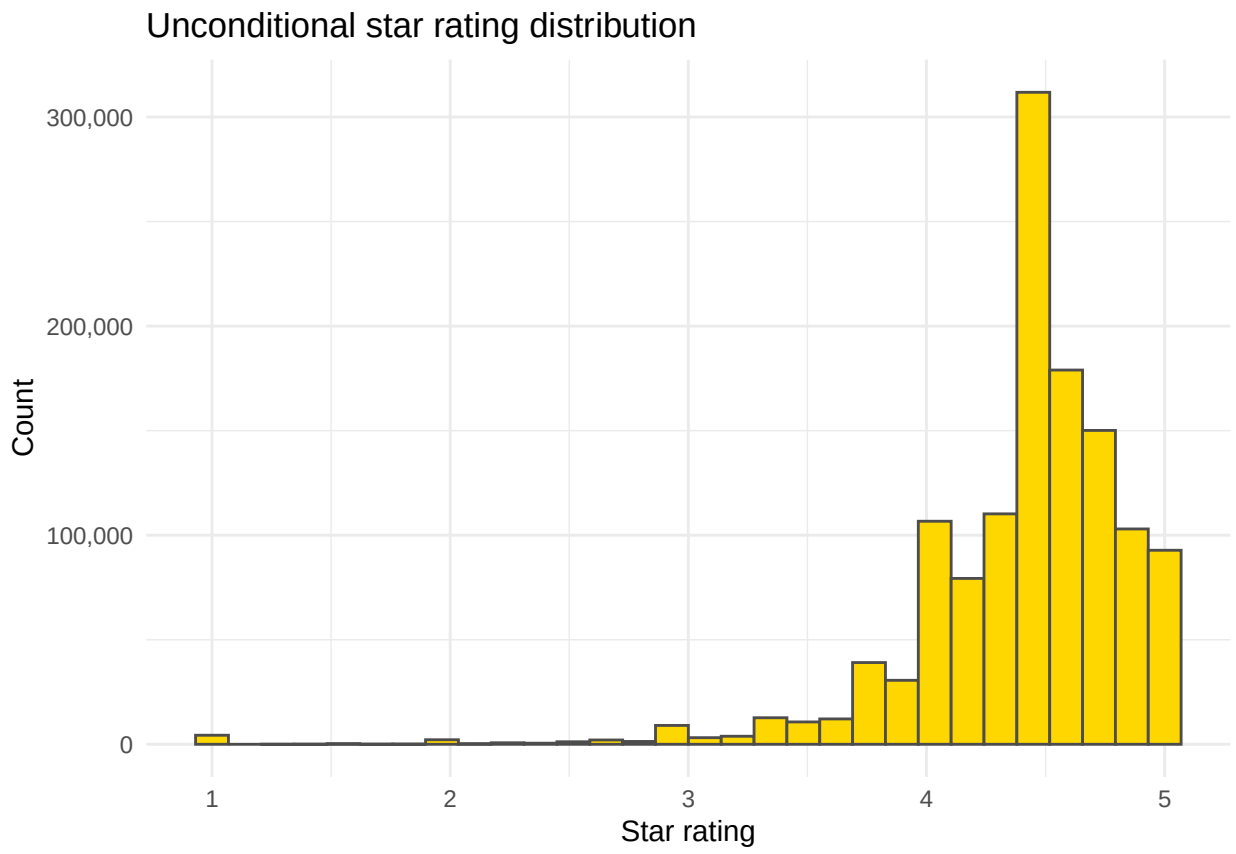


Best Seller density varies across categories by an order of magnitude, which is why `category_name` is kept as a high-dimensional fixed effect rather than collapsed. The boxplot below shows that Best Sellers cluster near the upper boundary of the rating scale, but their distributional overlap with non-Best Sellers is large; stars alone separate the classes only weakly.

```
products %>%
  mutate(label = ifelse(isBestSeller == 1, "Best Seller", "Other")) %>%
  ggplot(aes(label, stars, fill = label)) +
  geom_boxplot(outlier.alpha = 0.2) +
  labs(title = "Star rating by class",
       x = NULL, y = "Stars") +
  theme_minimal() +
  theme(legend.position = "none")
```

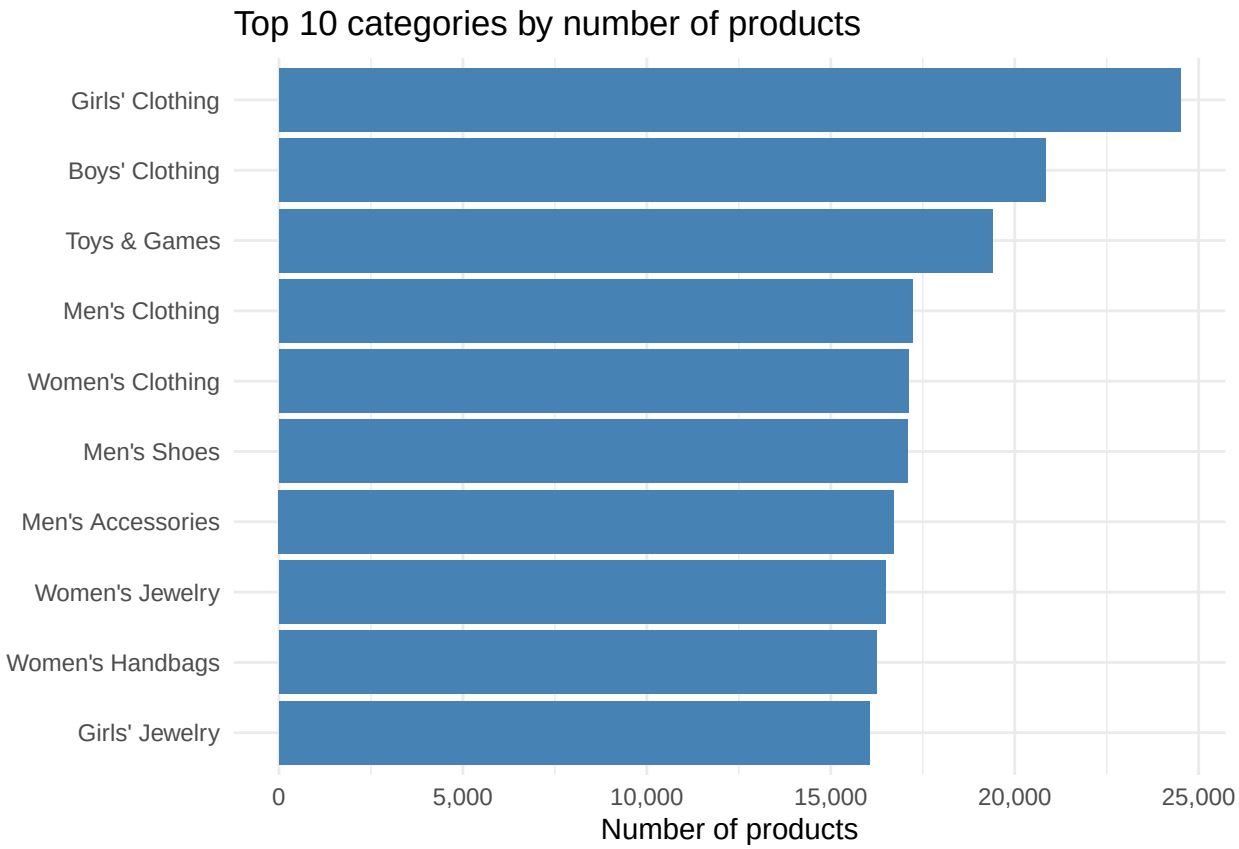



```
ggplot(products, aes(stars)) +
  geom_histogram(bins = 30, fill = "gold", color = "grey30") +
  scale_y_continuous(labels = comma) +
  labs(title = "Unconditional star rating distribution",
       x = "Star rating", y = "Count") +
  theme_minimal()
```



The unconditional rating distribution concentrates near 4.5 with a long lower tail. A Gaussian working model for `stars` is ruled out by this shape, and `stars` is therefore used as a continuous predictor with a small linear coefficient rather than as a target of inferential interest.

```
top_count <- products %>%
  count(category_name, sort = TRUE) %>%
  slice_head(n = 10)

ggplot(top_count, aes(reorder(category_name, n), n)) +
  geom_col(fill = "steelblue") +
  coord_flip() +
  scale_y_continuous(labels = comma) +
  labs(title = "Top 10 categories by number of products",
       x = NULL, y = "Number of products") +
  theme_minimal()
```



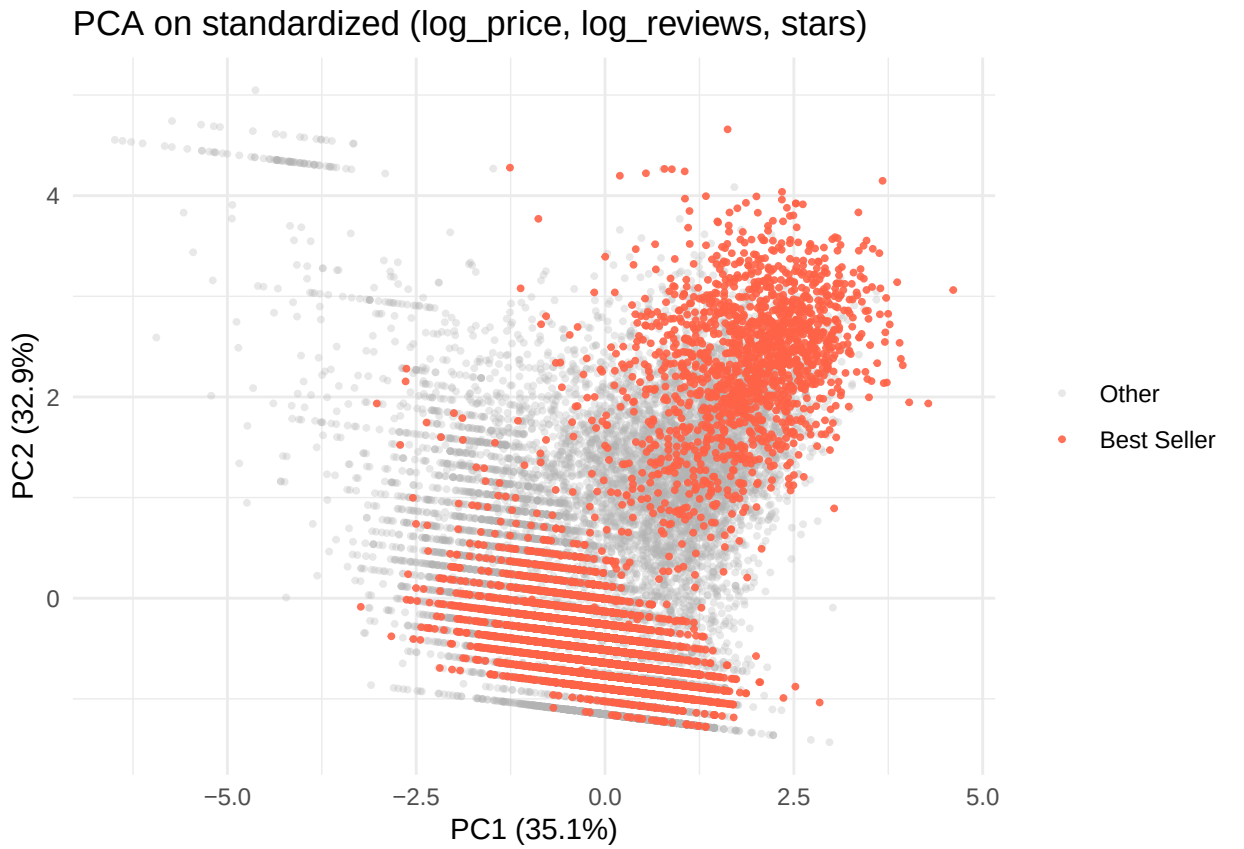
The top of the count distribution mixes apparel and a few specialty groups: the 3 largest categories are Girls' Clothing, Boys' Clothing, and Toys & Games, together accounting for about 5% of the analytic sample. The contrast with the Best Seller share ranking above is informative: the most populated categories are not the same as the categories with the highest Best Seller density, so category effects in the Lasso must be interpreted relative to a non-uniform baseline mix rather than a single dominant reference.

PCA on Numeric Predictors

A two-component PCA on standardized (`log_price, log_reviews, stars`) summarizes the geometry of the numeric predictor space.

```
num_mat <- products %>% select(log_price, log_reviews, stars) %>% scale()
pca <- prcomp(num_mat, center = FALSE, scale. = FALSE)
var_explained <- pca$sdev^2 / sum(pca$sdev^2)

set.seed(1)
plot_idx <- c(
  which(products$isBestSeller == 1),
  sample(which(products$isBestSeller == 0), size = 0.02 * sum(products$isBestSeller == 0))
)

tibble(
  PC1 = pca$x[plot_idx, 1],
  PC2 = pca$x[plot_idx, 2],
  cls = factor(products$isBestSeller[plot_idx], labels = c("Other", "Best Seller"))
```

```
) %>%
  arrange(cls) %>%
  ggplot(aes(PC1, PC2, color = cls, alpha = cls)) +
  geom_point(size = 0.7) +
  scale_color_manual(values = c("grey70", "tomato")) +
  scale_alpha_manual(values = c(0.3, 0.9)) +
  labs(title = "PCA on standardized (log_price, log_reviews, stars)",
       color = NULL, alpha = NULL,
       x = sprintf("PC1 (%.1f%%)", 100 * var_explained[1]),
       y = sprintf("PC2 (%.1f%%)", 100 * var_explained[2])) +
  theme_minimal()
```



The first two components capture most of the variance in the numeric block, but the Best Seller cloud overlaps the central mass of the negative class. The positive class is not linearly separable in the numeric subspace, so the discriminative work has to come from the categorical structure and from non-linear effects on review volume.

Methodology

5 model fits are evaluated. The first two are both Logistic Lassos. The fine-grained version one-hot encodes the 248 small categories and keeps within-group resolution under L1 shrinkage. The coarse version replaces those dummies with the 18 big-category groups and serves as the linear-additive baseline that the multilevel logistic later builds on by adding a small-category random intercept. Both Lassos share the same penalized log-likelihood objective:

$$\hat{\beta} \;=\; \arg\min_{\beta} \left\{ -\frac{1}{n} \sum_{i=1}^{n} \Big[y_i \, \mathbf{x}_i^{\top} \beta - \log\big(1 + e^{\mathbf{x}_i^{\top} \beta}\big) \Big] + \lambda \, \|\beta\|_1 \right\},$$

with λ selected by 10-fold cross-validation under inverse-prevalence class weights and the intercept left unpenalized.

The third model is a Generalized Additive Model with smoothing splines. The motivation is the diminishing-returns pattern in marketplace visibility: the marginal lift of an additional review is large when total reviews are small and shrinks as the count grows. A linear-in-log specification cannot represent that saturation, while smoothing splines can. The GAM uses the canonical logit link,

$$\log \frac{\Pr(Y_i = 1 \mid \mathbf{x}_i)}{1 - \Pr(Y_i = 1 \mid \mathbf{x}_i)} \;=\; \beta_0 + f_1\big(\log(1 + \text{reviews}_i)\big) + f_2\big(\log(\text{price}_i)\big) + \beta_1 \, \text{stars}_i + \boldsymbol{\gamma}_{g[i]},$$

where f_1 and f_2 are penalized regression splines fit by REML and $\boldsymbol{\gamma}$ is a parametric fixed effect on the coarse `big_category` grouping. The coarse grouping is preferred over the 248 small categories here so that the parametric block stays low-dimensional and the cross-group baseline differences from the unconditional EDA are still represented.

The fourth model is a Multilevel (Hierarchical) Logistic Regression. The Lasso treats the 248 categories as exchangeable one-hot dummies and ignores the natural nesting in the catalog. For example, "Boys' Clothing" and "Girls' Clothing" both belong to a broader Apparel group, and "Nintendo Switch" and "PlayStation 5 Consoles" both belong to Video Games. A multilevel logistic with a fixed effect for the coarse `big_category` grouping and a random intercept for the fine-grained `category_name` lets the small-cell categories shrink toward their parent group's mean rather than be estimated independently. The model is

$$\text{logit}\, \Pr(Y_{ij} = 1 \mid \mathbf{x}_{ij}) \;=\; \beta_0 + \beta_1 \, \log\big(\text{price}_{ij}\big) + \beta_2 \, \log\big(1 + \text{reviews}_{ij}\big) + \beta_3 \, \text{stars}_{ij} + \boldsymbol{\gamma}_{g[j]} + u_j, \quad u_j \sim \mathcal{N}(0, \sigma_u^2),$$

where j indexes the small category, $g[j]$ its big-category parent, $\boldsymbol{\gamma}$ are fixed effects on the parent groups, and u_j is the small category random intercept.

The fifth model is a Support Vector Machine with an RBF (Gaussian) kernel. The Lasso is linear in the predictors and the GAM treats each numeric variable additively. An RBF SVM allows arbitrary non-linear interaction between `log_price`, `log_reviews`, `stars`, and the categorical signal without a pre-specified functional form. Because RBF distances over a sparse one-hot encoding of 248 categories are degenerate, the SVM is fit on the standardized numeric block plus one-hot dummies for the coarse `big_category` grouping. The fine-grained categorical signal is left to the fine-grained Lasso and the multilevel logistic, where it fits more naturally. The RBF SVM solves

$$\min_{\mathbf{w}, b, \xi} \; \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^{n} \xi_i \quad \text{s.t.} \quad y_i\big(\mathbf{w}^{\top} \phi(\mathbf{x}_i) + b\big) \geq 1 - \xi_i, \;\; \xi_i \geq 0,$$

with ϕ implicit in the Gaussian kernel $K(\mathbf{x}, \mathbf{x}') = \exp(-\gamma \, \|\mathbf{x} - \mathbf{x}'\|^2)$ and class weights inversely proportional to the training prevalence. Probability outputs are obtained by Platt scaling.

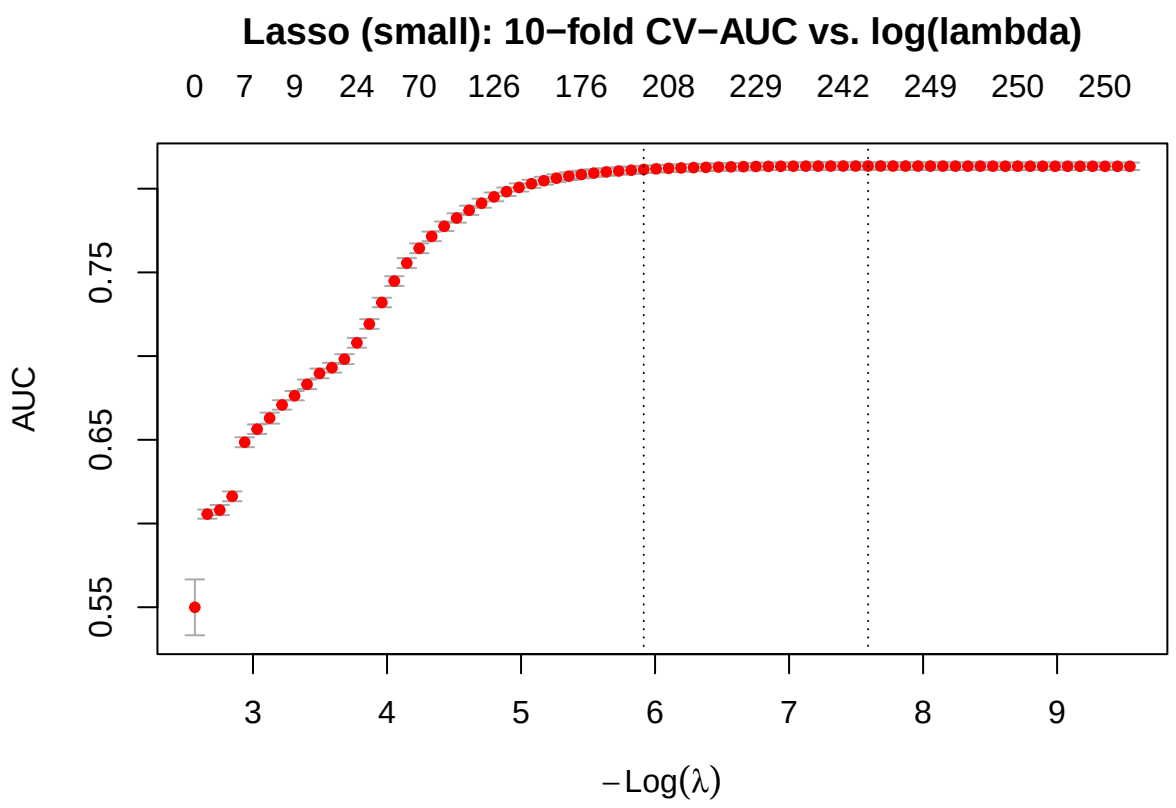
The PCA in Section 2 is exploratory: it maps the geometry of the numeric block and motivates including the categorical predictor in the supervised models. It does not feed engineered features into the supervised models, all of which operate directly on the original predictors.

Statistical Analysis

```
set.seed(1)
idx_pos <- which(products$isBestSeller == 1)
idx_neg <- which(products$isBestSeller == 0)
train_idx <- c(
  sample(idx_pos, size = floor(0.7 * length(idx_pos))),
  sample(idx_neg, size = floor(0.7 * length(idx_neg)))
)
train <- products[train_idx, ]
test  <- products[-train_idx, ]

w_train <- ifelse(
  train$isBestSeller == 1,
  1 / mean(train$isBestSeller),
  1 / (1 - mean(train$isBestSeller))
)
```

Model 1a: Logistic Lasso - Fine-Grained Category (248 category)

```
X_full <- sparse.model.matrix(
  ~ log_price + log_reviews + stars + category_name - 1,
  data = products
)
X_train <- X_full[train_idx,  , drop = FALSE]
X_test  <- X_full[-train_idx, , drop = FALSE]
y_train <- train$isBestSeller
y_test  <- test$isBestSeller

set.seed(1)
cv_lasso <- cv.glmnet(
  X_train, y_train,
  family = "binomial",
  weights = w_train,
  type.measure = "auc",
  nfolds = 10
)
plot(cv_lasso)
title("Lasso (small): 10-fold CV-AUC vs. log(lambda)", line = 2.6)
```




```
p_lasso <- as.numeric(predict(cv_lasso, X_test, s = "lambda.min", type = "response"))
```

```
coefs <- as.matrix(coef(cv_lasso, s = "lambda.min"))
nz <- data.frame(term = rownames(coefs), coef = coefs[, 1]) %>%
  filter(term != "(Intercept)", coef != 0) %>%
  arrange(desc(abs(coef))) %>%
  head(10)
nz
```

```
##
## category_nameNintendo Switch Consoles, Games & Accessories category_nameNintendo Switch Consoles, Ga
## category_nameBuilding Toys                                                                  category_
## category_nameOnline Video Game Services                                                 category_nameOnline Vi
## category_nameKids' Play Cars & Race Cars                                                category_nameKids' Play
## category_nameLaptop Bags                                                                          categor
## category_nameHeadphones & Earbuds                                                           category_nameHea
## category_nameGPS & Navigation                                                                category_nam
## category_nameTools & Home Improvement                                                   category_nameTools &
## category_nameWomen's Watches                                                                category_na
## category_nameLight Bulbs                                                                          categor
##                                                                 coef
## category_nameNintendo Switch Consoles, Games & Accessories -4.441383
## category_nameBuilding Toys                                 -3.927449
## category_nameOnline Video Game Services                     3.804630
## category_nameKids' Play Cars & Race Cars                   -3.796093
```

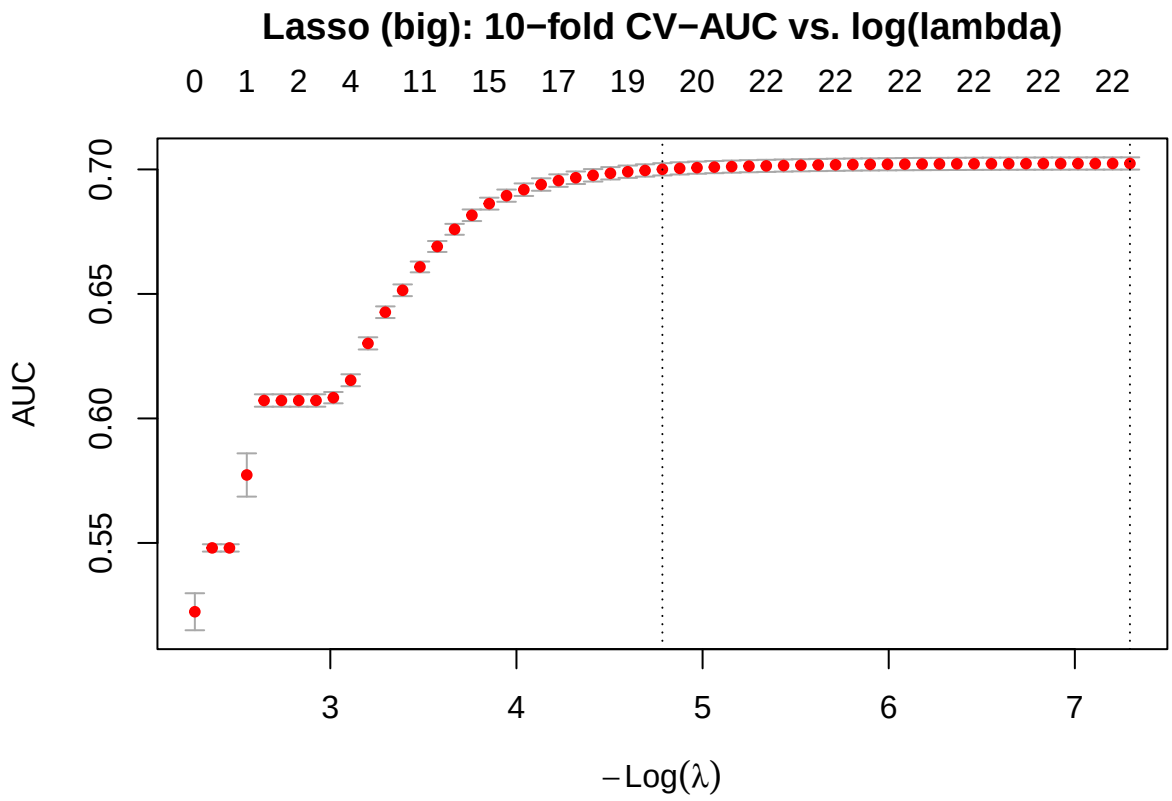
```
## category_nameLaptop Bags                                -3.755985
## category_nameHeadphones & Earbuds                       -3.728836
## category_nameGPS & Navigation                           -3.715296
## category_nameTools & Home Improvement                    3.703654
## category_nameWomen's Watches                            -3.686254
## category_nameLight Bulbs                                -3.667643
```

The L1 penalty produces shrinkage but does not push the solution toward sparsity here. The selected `lambda.min` keeps most category dummies and drives only a small number exactly to zero. The continuous predictors all carry non-zero coefficients. In absolute magnitude, `stars` is the strongest, `log_reviews` second, and `log_price` smallest and negative. Within the linear-additive specification the rating signal therefore dominates review volume, and the category effects are spread across most of the levels rather than being concentrated in a handful.

Model 1b: Logistic Lasso - Coarse Category (18 big_category)

The fine-grained Lasso treats the 248 small categories as exchangeable. Replacing `category_name` with `big_category` collapses the hierarchy to its top level: every small category contributes only through its parent group's coefficient. This variant trades within-group resolution for interpretability and stability, and serves as the linear-additive counterpart to the multilevel logistic that follows.

```
X_full_big <- sparse.model.matrix(
  ~ log_price + log_reviews + stars + big_category - 1,
  data = products
)
X_train_big <- X_full_big[train_idx,  , drop = FALSE]
X_test_big  <- X_full_big[-train_idx, , drop = FALSE]

set.seed(1)
cv_lasso_big <- cv.glmnet(
  X_train_big, y_train,
  family       = "binomial",
  weights      = w_train,
  type.measure = "auc",
  nfolds       = 10
)
plot(cv_lasso_big)
title("Lasso (big): 10-fold CV-AUC vs. log(lambda)", line = 2.6)
```




```
p_lasso_big <- as.numeric(predict(cv_lasso_big, X_test_big, s = "lambda.min", type = "response"))

coefs_big <- as.matrix(coef(cv_lasso_big, s = "lambda.min"))
nz_big <- data.frame(term = rownames(coefs_big), coef = coefs_big[, 1]) %>%
  filter(term != "(Intercept)", coef != 0) %>%
  arrange(desc(abs(coef)))
nz_big
```

```
##                                                                  term
## big_categorySports & Outdoor                 big_categorySports & Outdoor
## big_categoryWearable & VR                       big_categoryWearable & VR
## big_categoryOffice & Party                     big_categoryOffice & Party
## big_categoryLuggage & Travel                 big_categoryLuggage & Travel
## big_categoryAutomotive                             big_categoryAutomotive
## big_categorySmart Home                             big_categorySmart Home
## stars                                                               stars
## big_categoryHealth & Wellness               big_categoryHealth & Wellness
## big_categoryArts & Crafts                       big_categoryArts & Crafts
## big_categoryHhousehold Supplies           big_categoryHhousehold Supplies
## big_categoryTools & Home Improvement big_categoryTools & Home Improvement
## big_categoryVideo Games                           big_categoryVideo Games
## big_categoryIndustrial & Scientific   big_categoryIndustrial & Scientific
## big_categoryToys & Games                         big_categoryToys & Games
## big_categoryElectronics                           big_categoryElectronics
## big_categoryBaby & Maternity                 big_categoryBaby & Maternity
```

```
## big_categoryHome & Kitchen                    big_categoryHome & Kitchen
## big_categoryApparel & Accessories       big_categoryApparel & Accessories
## log_price                                                     log_price
## big_categoryOther                                     big_categoryOther
## big_categoryBeauty & Personal Care     big_categoryBeauty & Personal Care
## log_reviews                                                 log_reviews
##                                                                    coef
## big_categorySports & Outdoor                                2.60038780
## big_categoryWearable & VR                                  -2.33223689
## big_categoryOffice & Party                                 -1.61283280
## big_categoryLuggage & Travel                               -1.59821957
## big_categoryAutomotive                                      1.04327926
## big_categorySmart Home                                      1.02401006
## stars                                                       0.95414755
## big_categoryHealth & Wellness                               0.67267547
## big_categoryArts & Crafts                                  -0.64403785
## big_categoryHousehold Supplies                              0.56745795
## big_categoryTools & Home Improvement                        0.54498516
## big_categoryVideo Games                                    -0.53541663
## big_categoryIndustrial & Scientific                         0.49143178
## big_categoryToys & Games                                   -0.46225978
## big_categoryElectronics                                    -0.35628211
## big_categoryBaby & Maternity                               -0.22639756
## big_categoryHome & Kitchen                                  0.22393129
## big_categoryApparel & Accessories                          -0.22075118
## log_price                                                  -0.16721620
## big_categoryOther                                           0.15695993
## big_categoryBeauty & Personal Care                         -0.14949646
## log_reviews                                                 0.08809478
```

With only 18 group dummies the L1 penalty does little: each big-category coefficient is identified by tens of thousands of observations, so almost every group enters the model with a non-zero estimate. Comparing this fit against the fine-grained Lasso quantifies the cost of collapsing 248 levels into 18 groups; any drop in PR-AUC is signal lost to the coarsening. This coarse Lasso is the fixed-effect-only baseline that the multilevel model later builds on by adding a small-category random intercept.

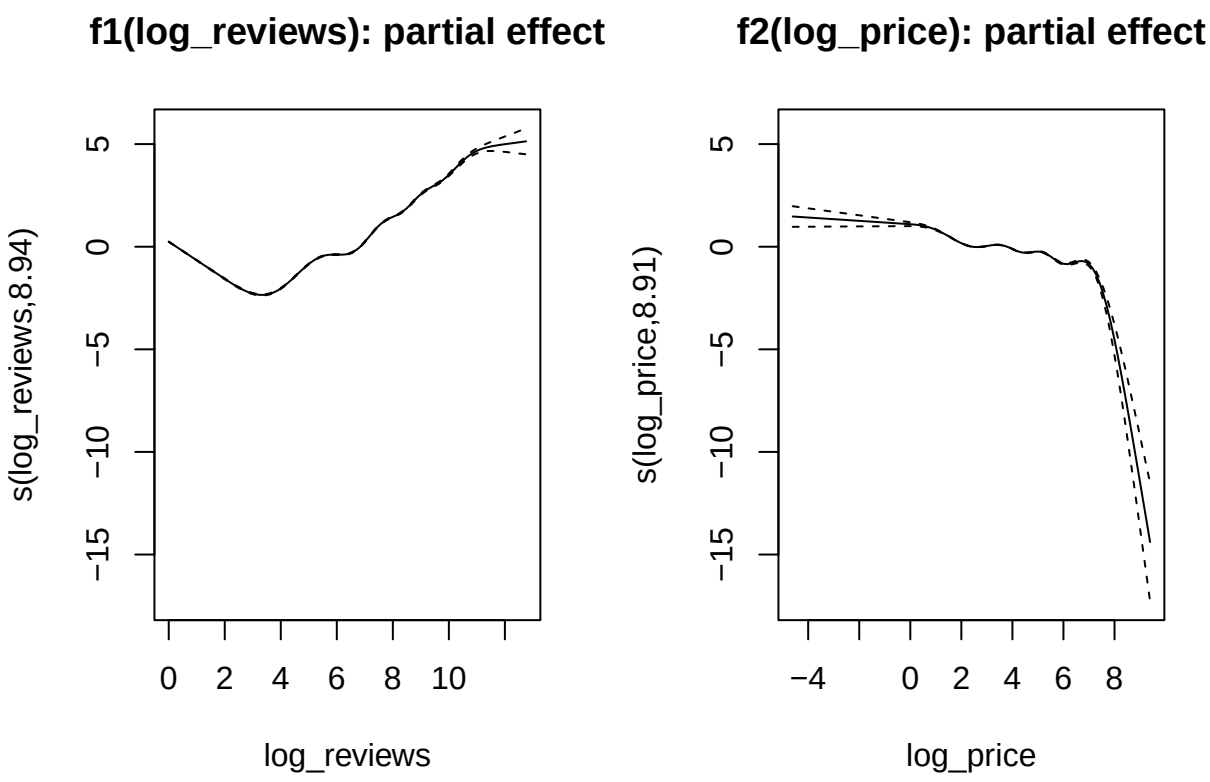
Model 2: Generalized Additive Model (with big_category)

```r
gam_fit <- gam(
  isBestSeller ~ s(log_reviews) + s(log_price) + stars + big_category,
  family  = binomial,
  data    = train,
  method  = "REML",
  weights = w_train
)
summary(gam_fit)
```

```
##
## Family: binomial
## Link function: logit
##
```

```
## Formula:
## isBestSeller ~ s(log_reviews) + s(log_price) + stars + big_category
##
## Parametric coefficients:
##                                      Estimate Std. Error  z value Pr(>|z|)
## (Intercept)                         -4.506777   0.024935 -180.741  < 2e-16 ***
## stars                                0.881391   0.005500  160.240  < 2e-16 ***
## big_categoryArts & Crafts           -0.422662   0.009572  -44.157  < 2e-16 ***
## big_categoryAutomotive               1.290781   0.008966  143.963  < 2e-16 ***
## big_categoryBaby & Maternity        -0.079034   0.011365   -6.954 3.55e-12 ***
## big_categoryBeauty & Personal Care  -0.123065   0.009877  -12.460  < 2e-16 ***
## big_categoryElectronics             -0.191481   0.007111  -26.928  < 2e-16 ***
## big_categoryHealth & Wellness        0.800697   0.009296   86.137  < 2e-16 ***
## big_categoryHome & Kitchen           0.393060   0.005886   66.783  < 2e-16 ***
## big_categoryHousehold Supplies       0.656273   0.015911   41.245  < 2e-16 ***
## big_categoryIndustrial & Scientific  0.685093   0.006067  112.927  < 2e-16 ***
## big_categoryLuggage & Travel        -1.559493   0.018017  -86.556  < 2e-16 ***
## big_categoryOffice & Party          -1.467994   0.022133  -66.325  < 2e-16 ***
## big_categoryOther                    0.236457   0.011900   19.870  < 2e-16 ***
## big_categoryPet Supplies             0.117557   0.010590   11.101  < 2e-16 ***
## big_categorySmart Home               1.168880   0.037575   31.108  < 2e-16 ***
## big_categorySports & Outdoor         2.692885   0.011992  224.559  < 2e-16 ***
## big_categoryTools & Home Improvement 0.755814   0.006530  115.753  < 2e-16 ***
## big_categoryToys & Games            -0.321210   0.007982  -40.243  < 2e-16 ***
## big_categoryVideo Games             -0.559370   0.012844  -43.550  < 2e-16 ***
## big_categoryWearable & VR           -1.876320   0.050937  -36.836  < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Approximate significance of smooth terms:
##                   edf Ref.df Chi.sq p-value
## s(log_reviews) 8.940  8.998  94890  <2e-16 ***
## s(log_price)   8.906  8.993   9626  <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## R-sq.(adj) =  0.0888   Deviance explained = 16.7%
## -REML = 1.0246e+06  Scale est. = 1         n = 887030
```

```r
par(mfrow = c(1, 2))
plot(gam_fit, select = 1, shade = TRUE, seWithMean = TRUE,
     main = "f1(log_reviews): partial effect")
plot(gam_fit, select = 2, shade = TRUE, seWithMean = TRUE,
     main = "f2(log_price): partial effect")
```




```
par(mfrow = c(1, 1))
```

The estimated $f_1(\log(1 + \text{reviews}))$ rises over the lower half of the support and flattens for high-review products, the diminishing-returns pattern anticipated in Methodology. The smooth on `log_price` is approximately monotone-decreasing, indicating that lower-priced products are more likely to reach Best Seller status.

```
edf_tbl <- tibble(
  Smooth = rownames(summary(gam_fit)$s.table),
  EDF    = summary(gam_fit)$s.table[, "edf"],
  `Ref df` = summary(gam_fit)$s.table[, "Ref.df"],
  `p-value` = summary(gam_fit)$s.table[, "p-value"]
)
as.data.frame(edf_tbl)
```

```
##           Smooth      EDF   Ref df p-value
## 1 s(log_reviews) 8.940026 8.998322       0
## 2   s(log_price) 8.905986 8.992557       0
```

The effective degrees of freedom quantify how non-linear each smooth actually is: an EDF near 1 collapses to a linear term, while values above 3 indicate genuine curvature that a polynomial of low order cannot reproduce. The EDF reported for $f_1(\log(1 + \text{reviews}))$ sits well above unity, consistent with the saturation shape, and the Wald-style p-value rejects the null of a flat smooth at any conventional level. The smooth on log(price) also exceeds linearity, justifying its inclusion as a non-parametric term rather than a single coefficient.

```r
p_gam <- as.numeric(predict(gam_fit, newdata = test, type = "response"))
```

Model 3: Multilevel (Hierarchical) Logistic Regression

The model couples a fixed effect on a coarse `big_category` grouping with a random intercept on the fine-grained `category_name`. The grouping is built from regular-expression rules over the 248 category labels and produces parent groups such as Apparel, Electronics, Toys & Games, Home & Kitchen, Tools & Home Improvement, and Industrial & Scientific. Fitting on the full training set with 248 random levels is expensive, so a stratified subsample is used: up to 200 rows per category, plus every positive instance from the training fold. This sampling scheme keeps every category in the design matrix and keeps every positive instance in the training set.

```r
set.seed(1)
ml_strat <- train %>%
  mutate(.row = row_number()) %>%
  group_by(category_name) %>%
  slice_sample(n = 200) %>%
  ungroup() %>%
  pull(.row)
ml_pos <- which(train$isBestSeller == 1)
ml_idx <- unique(c(ml_strat, ml_pos))
train_ml <- train[ml_idx, ]

ml_fit <- glmer(
  isBestSeller ~ log_price + log_reviews + stars + big_category +
                 (1 | category_name),
  data    = train_ml,
  family  = binomial(),
  control = glmerControl(optimizer = "bobyqa",
                         optCtrl   = list(maxfun = 2e5)),
  nAGQ    = 0
)

p_ml <- as.numeric(predict(ml_fit, newdata = test,
                           type = "response",
                           allow.new.levels = TRUE))
```

```r
ml_fixed <- broom.mixed::tidy(ml_fit, effects = "fixed") %>%
  select(term, estimate, std.error, p.value) %>%
  mutate(across(where(is.numeric), ~ round(., 4)))
ml_var <- as.data.frame(VarCorr(ml_fit))$vcov
data.frame(
  metric = c("Fixed effects (rows)", "big_category levels",
             "category_name (random) variance",
             "category_name (random) SD"),
  value  = c(nrow(ml_fixed),
             length(unique(train_ml$big_category)),
             sprintf("%.4f", ml_var[1]),
             sprintf("%.4f", sqrt(ml_var[1])))
) %>% as.data.frame()
```

```
##                           metric  value
```

```
## 1            Fixed effects (rows)     23
## 2             big_category levels     20
## 3 category_name (random) variance 2.0588
## 4       category_name (random) SD 1.4348
```

```
ml_fixed
```

```
## # A tibble: 23 x 4
##    term                                estimate std.error p.value
##    <chr>                                  <dbl>     <dbl>   <dbl>
##  1 (Intercept)                           -5.93     0.352   0
##  2 log_price                             -0.108    0.0203  0
##  3 log_reviews                            0.356    0.0124  0
##  4 stars                                  0.721    0.0547  0
##  5 big_categoryArts & Crafts             -0.501    0.551   0.363
##  6 big_categoryAutomotive                 0.358    0.662   0.589
##  7 big_categoryBaby & Maternity          -0.722    0.536   0.178
##  8 big_categoryBeauty & Personal Care    -0.386    0.553   0.485
##  9 big_categoryElectronics               -1.04     0.397   0.0089
## 10 big_categoryHealth & Wellness          0.464    0.52    0.372
## # i 13 more rows
```

The variance of the small-category random intercept measures residual heterogeneity across categories after the big-category fixed effects have absorbed the gross structure. The fixed-effect coefficients on the continuous predictors carry the same sign as in the Lasso fit, which is a useful internal check on the linear-additive evidence.

Model 4: Support Vector Machine - RBF Kernel (with `big_category`)

The RBF SVM is fit on a stratified subsample of the training set: all positives plus a 5:1 ratio of negatives. Subsampling is required because the kernel matrix scales quadratically with the training size and an unsubsampled fit on 887,030 rows is computationally expensive. The 5:1 negative-to-positive ratio preserves the relative class weighting argument while keeping the optimization tractable.

```
set.seed(1)
sub_pos <- which(train$isBestSeller == 1)
sub_neg <- sample(which(train$isBestSeller == 0), size = 5 * length(sub_pos))
sub_idx <- c(sub_pos, sub_neg)
train_sub <- train[sub_idx, ]

sub_prev    <- mean(train_sub$isBestSeller)
svm_weights <- c("0" = 1 / (1 - sub_prev), "1" = 1 / sub_prev)

svm_fit <- svm(
  factor(isBestSeller) ~ log_price + log_reviews + stars + big_category,
  data           = train_sub,
  kernel         = "radial",
  cost           = 1,
  probability    = TRUE,
  scale          = TRUE,
  class.weights  = svm_weights
)
```

```r
svm_pred <- predict(svm_fit, newdata = test, probability = TRUE)
p_svm    <- attr(svm_pred, "probabilities")[, "1"]
```

```r
data.frame(
  metric = c("Subsample size", "Subsample BS prevalence",
             "Support vectors",
             "Numeric features", "big_category dummies"),
  value  = c(comma(nrow(train_sub)),
             percent(sub_prev, accuracy = 0.01),
             comma(svm_fit$tot.nSV),
             3,
             length(unique(train_sub$big_category)) - 1)
) %>% as.data.frame()
```

```
##                    metric  value
## 1          Subsample size 35,406
## 2 Subsample BS prevalence 16.67%
## 3         Support vectors 23,520
## 4        Numeric features      3
## 5    big_category dummies     19
```

The number of support vectors gives a rough measure of how complex the decision boundary is in the kernel feature space. The feature set combines the 3 standardized numeric predictors with one-hot dummies for the 18 big-category groups. Using the 248 small categories directly would degenerate the RBF distance, because the sparse one-hot dimensions would overwhelm the 3 numeric ones; the coarse grouping is the natural compromise between including category signal and preserving kernel geometry. Probability outputs are obtained by post-hoc Platt scaling, which is the quantity of interest for rare-event ranking.

Evaluation

Accuracy is uninformative here: a constant predictor that always returns zero achieves 99.33% accuracy and zero recall. The two metrics reported below are insensitive to this degeneracy. ROC-AUC measures the probability that a random positive is ranked above a random negative. PR-AUC summarizes the precision-recall trade-off and has the prevalence (0.0067) as its trivial baseline, which makes it the discriminating metric under extreme class imbalance.

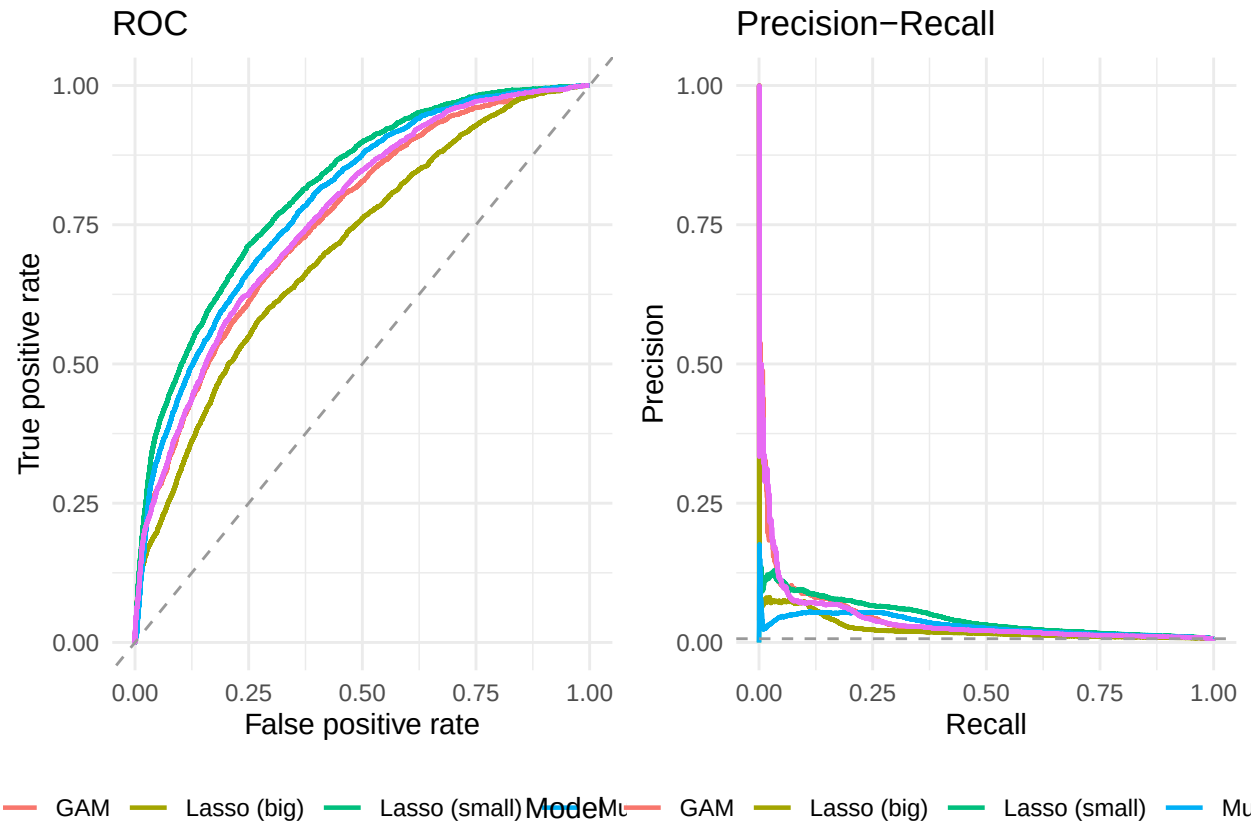
```r
roc_lasso     <- pROC::roc(y_test, p_lasso,     quiet = TRUE)
roc_lasso_big <- pROC::roc(y_test, p_lasso_big, quiet = TRUE)
roc_gam       <- pROC::roc(y_test, p_gam,       quiet = TRUE)
roc_ml        <- pROC::roc(y_test, p_ml,        quiet = TRUE)
roc_svm       <- pROC::roc(y_test, p_svm,       quiet = TRUE)

pr_lasso     <- pr.curve(scores.class0 = p_lasso[y_test == 1],
                         scores.class1 = p_lasso[y_test == 0], curve = TRUE)
pr_lasso_big <- pr.curve(scores.class0 = p_lasso_big[y_test == 1],
                         scores.class1 = p_lasso_big[y_test == 0], curve = TRUE)
pr_gam       <- pr.curve(scores.class0 = p_gam[y_test == 1],
                         scores.class1 = p_gam[y_test == 0], curve = TRUE)
pr_ml        <- pr.curve(scores.class0 = p_ml[y_test == 1],
                         scores.class1 = p_ml[y_test == 0], curve = TRUE)
pr_svm       <- pr.curve(scores.class0 = p_svm[y_test == 1],
```

```
                        scores.class1 = p_svm[y_test == 0], curve = TRUE)

eval_tbl <- tibble(
  Model   = c("Lasso (small)", "Lasso (big)", "GAM", "Multilevel", "SVM (RBF)"),
  `ROC-AUC` = c(as.numeric(auc(roc_lasso)),
                as.numeric(auc(roc_lasso_big)),
                as.numeric(auc(roc_gam)),
                as.numeric(auc(roc_ml)),
                as.numeric(auc(roc_svm))),
  `PR-AUC`  = c(pr_lasso$auc.integral,
                pr_lasso_big$auc.integral,
                pr_gam$auc.integral,
                pr_ml$auc.integral,
                pr_svm$auc.integral)
)
as.data.frame(eval_tbl)
```

```
##           Model   ROC-AUC     PR-AUC
## 1 Lasso (small) 0.8133730 0.04347700
## 2   Lasso (big) 0.7071961 0.02353347
## 3           GAM 0.7576618 0.04067882
## 4    Multilevel 0.7904705 0.02977658
## 5     SVM (RBF) 0.7662830 0.03965221
```

```
roc_df <- bind_rows(
  tibble(Model = "Lasso (small)", FPR = 1 - roc_lasso$specificities,     TPR = roc_lasso$sensitivities),
  tibble(Model = "Lasso (big)",   FPR = 1 - roc_lasso_big$specificities, TPR = roc_lasso_big$sensitivit
  tibble(Model = "GAM",           FPR = 1 - roc_gam$specificities,       TPR = roc_gam$sensitivities),
  tibble(Model = "Multilevel",    FPR = 1 - roc_ml$specificities,        TPR = roc_ml$sensitivities),
  tibble(Model = "SVM",           FPR = 1 - roc_svm$specificities,       TPR = roc_svm$sensitivities)
)

pr_df <- bind_rows(
  tibble(Model = "Lasso (small)", Recall = pr_lasso$curve[, 1],     Precision = pr_lasso$curve[, 2]),
  tibble(Model = "Lasso (big)",   Recall = pr_lasso_big$curve[, 1], Precision = pr_lasso_big$curve[, 2]
  tibble(Model = "GAM",           Recall = pr_gam$curve[, 1],       Precision = pr_gam$curve[, 2]),
  tibble(Model = "Multilevel",    Recall = pr_ml$curve[, 1],        Precision = pr_ml$curve[, 2]),
  tibble(Model = "SVM",           Recall = pr_svm$curve[, 1],       Precision = pr_svm$curve[, 2])
)

p_roc <- ggplot(roc_df, aes(FPR, TPR, color = Model)) +
  geom_line(linewidth = 0.8) +
  geom_abline(slope = 1, intercept = 0, linetype = "dashed", color = "grey60") +
  labs(title = "ROC", x = "False positive rate", y = "True positive rate") +
  theme_minimal() +
  theme(legend.position = "bottom")

p_pr <- ggplot(pr_df, aes(Recall, Precision, color = Model)) +
  geom_line(linewidth = 0.8) +
  geom_hline(yintercept = mean(y_test), linetype = "dashed", color = "grey60") +
  labs(title = "Precision-Recall",
       x = "Recall", y = "Precision") +
  theme_minimal() +
```

```
  theme(legend.position = "bottom")

gridExtra::grid.arrange(p_roc, p_pr, ncol = 2)
```



The ROC curves separate from the diagonal across most of the FPR range, but the practically relevant evidence is in the precision-recall panel: the dashed horizontal at the test prevalence (0.0067) is the baseline, and the 5 curves dominate it by a wide margin in the low-recall regime where rare-event ranking is operationally useful. The visual gap between models in the PR panel mirrors the PR-AUC margins reported in the table above.

```
prob_check <- tibble(
  Model = rep(c("Lasso (small)", "Lasso (big)", "GAM", "Multilevel", "SVM"), each = 2),
  Class = rep(c("Negatives (y=0)", "Positives (y=1)"), times = 5),
  `Median p` = c(
    median(p_lasso[y_test == 0]),     median(p_lasso[y_test == 1]),
    median(p_lasso_big[y_test == 0]), median(p_lasso_big[y_test == 1]),
    median(p_gam[y_test == 0]),       median(p_gam[y_test == 1]),
    median(p_ml[y_test == 0]),        median(p_ml[y_test == 1]),
    median(p_svm[y_test == 0]),       median(p_svm[y_test == 1])
  ),
  `95th pct p` = c(
    quantile(p_lasso[y_test == 0],     0.95), quantile(p_lasso[y_test == 1],     0.95),
    quantile(p_lasso_big[y_test == 0], 0.95), quantile(p_lasso_big[y_test == 1], 0.95),
    quantile(p_gam[y_test == 0],       0.95), quantile(p_gam[y_test == 1],       0.95),
    quantile(p_ml[y_test == 0],        0.95), quantile(p_ml[y_test == 1],        0.95),
    quantile(p_svm[y_test == 0],       0.95), quantile(p_svm[y_test == 1],       0.95)
```

```
  )
)
as.data.frame(prob_check)
```

```
##            Model           Class   Median p 95th pct p
## 1  Lasso (small) Negatives (y=0) 0.33415966  0.7480480
## 2  Lasso (small) Positives (y=1) 0.64558326  0.9469264
## 3    Lasso (big) Negatives (y=0) 0.43064480  0.6914496
## 4    Lasso (big) Positives (y=1) 0.56655861  0.9191654
## 5            GAM Negatives (y=0) 0.40493934  0.7032489
## 6            GAM Positives (y=1) 0.59063806  0.9347746
## 7     Multilevel Negatives (y=0) 0.08050508  0.3562204
## 8     Multilevel Positives (y=1) 0.24348541  0.6590871
## 9            SVM Negatives (y=0) 0.08511117  0.3571041
## 10           SVM Positives (y=1) 0.32549507  0.5171395
```

Under inverse-prevalence weighting, the median predicted probability for true negatives stays close to the empirical prevalence, the median for true positives is roughly an order of magnitude higher, and the 95th percentile of the positive class is well above any reasonable operating threshold. No collapse to a degenerate constant is observed, which is what the weighted likelihood is meant to prevent.

Operating-point metrics for rare events

ROC-AUC and PR-AUC summarize ranking quality across all thresholds. At 0.67% prevalence, however, a sharper question is more useful: *if a curator can review only the top K% of products, how many true Best Sellers are captured, and at what precision?* The metrics below answer this directly. Precision@K is the fraction of correct positives among the top K% of predicted scores. Recall@K is the fraction of true positives that fall in that top slice. Specificity@K is the share of true negatives correctly kept out. Lift@K is precision divided by the prevalence baseline.

```
prev_test <- mean(y_test == 1)

rare_event_metrics <- function(p, y, ks = c(0.005, 0.01, 0.05, 0.10)) {
  N         <- length(p)
  total_pos <- sum(y == 1)
  total_neg <- sum(y == 0)
  prev      <- mean(y == 1)
  purrr::map_df(ks, function(k) {
    n_top  <- ceiling(k * N)
    thr    <- sort(p, decreasing = TRUE)[n_top]
    flag   <- p >= thr
    tp     <- sum(flag & y == 1)
    fp     <- sum(flag & y == 0)
    tn     <- sum(!flag & y == 0)
    tibble(
      `Top K`       = scales::percent(k, accuracy = 0.1),
      Precision     = tp / max(sum(flag), 1),
      Recall        = tp / total_pos,
      Specificity   = tn / total_neg,
      Lift          = (tp / max(sum(flag), 1)) / prev
    )
  })
```

```
}

operating_tbl <- bind_rows(
  rare_event_metrics(p_lasso,     y_test) %>% mutate(Model = "Lasso (small)"),
  rare_event_metrics(p_lasso_big, y_test) %>% mutate(Model = "Lasso (big)"),
  rare_event_metrics(p_gam,       y_test) %>% mutate(Model = "GAM"),
  rare_event_metrics(p_ml,        y_test) %>% mutate(Model = "Multilevel"),
  rare_event_metrics(p_svm,       y_test) %>% mutate(Model = "SVM")
) %>%
  mutate(across(c(Precision, Recall, Specificity), ~ round(., 4)),
         Lift = round(Lift, 1)) %>%
  select(Model, `Top K`, Precision, Recall, Specificity, Lift)

as.data.frame(operating_tbl)
```

```
##            Model Top K Precision Recall Specificity Lift
## 1  Lasso (small)  0.5%    0.0956 0.0720      0.9954 14.4
## 2  Lasso (small)  1.0%    0.0857 0.1289      0.9908 12.9
## 3  Lasso (small)  5.0%    0.0505 0.3792      0.9522  7.6
## 4  Lasso (small) 10.0%    0.0325 0.4887      0.9026  4.9
## 5    Lasso (big)  0.5%    0.0745 0.0561      0.9953 11.2
## 6    Lasso (big)  1.0%    0.0710 0.1068      0.9906 10.7
## 7    Lasso (big)  5.0%    0.0266 0.2001      0.9510  4.0
## 8    Lasso (big) 10.0%    0.0204 0.3060      0.9014  3.1
## 9            GAM  0.5%    0.0978 0.0735      0.9955 14.7
## 10           GAM  1.0%    0.0815 0.1226      0.9908 12.3
## 11           GAM  5.0%    0.0364 0.2736      0.9515  5.5
## 12           GAM 10.0%    0.0255 0.3828      0.9019  3.8
## 13    Multilevel  0.5%    0.0326 0.0245      0.9951  4.9
## 14    Multilevel  1.0%    0.0499 0.0751      0.9904  7.5
## 15    Multilevel  5.0%    0.0432 0.3250      0.9518  6.5
## 16    Multilevel 10.0%    0.0295 0.4433      0.9023  4.4
## 17           SVM  0.5%    0.0857 0.0645      0.9954 12.9
## 18           SVM  1.0%    0.0705 0.1060      0.9906 10.6
## 19           SVM  5.0%    0.0366 0.2748      0.9515  5.5
## 20           SVM 10.0%    0.0255 0.3839      0.9019  3.8
```

The Top-1% slice is the most useful row for a curator with a small inspection budget. A Best Seller hit rate above the 0.67% baseline at this slice means that the model concentrates the rare positive class into a narrow head of the ranking. Lift quantifies this concentration as a multiplier of the baseline, and recall states the share of all Best Sellers captured at that operating point. Specificity, the complement of the false-positive rate, stays near one across all slices because the negative class dominates the sample, but its value at each K is reported for completeness.

Model Comparison

```
ord_pr   <- order(-eval_tbl$`PR-AUC`)
ord_roc  <- order(-eval_tbl$`ROC-AUC`)
winner_pr  <- eval_tbl$Model[ord_pr[1]]
runner_pr  <- eval_tbl$Model[ord_pr[2]]
delta_pr   <- eval_tbl$`PR-AUC`[ord_pr[1]] - eval_tbl$`PR-AUC`[ord_pr[2]]
```

```r
winner_roc <- eval_tbl$Model[ord_roc[1]]
delta_roc  <- eval_tbl$`ROC-AUC`[ord_roc[1]] - eval_tbl$`ROC-AUC`[ord_roc[2]]
baseline   <- mean(y_test)
lift_pr    <- max(eval_tbl$`PR-AUC`) / baseline

comparison_tbl <- tibble(
  Criterion = c(
    "PR-AUC winner",
    "PR-AUC runner-up",
    "PR-AUC margin (1st - 2nd)",
    "ROC-AUC winner",
    "ROC-AUC margin (1st - 2nd)",
    "Test-set prevalence (PR-AUC baseline)",
    "PR-AUC lift over baseline",
    "Recommended model"
  ),
  Value = c(
    winner_pr,
    runner_pr,
    sprintf("%.4f", delta_pr),
    winner_roc,
    sprintf("%.4f", delta_roc),
    sprintf("%.4f", baseline),
    sprintf("%.1fx", lift_pr),
    winner_pr
  )
)
as.data.frame(comparison_tbl)
```

```
##                              Criterion         Value
## 1                        PR-AUC winner Lasso (small)
## 2                       PR-AUC runner-up           GAM
## 3             PR-AUC margin (1st - 2nd)        0.0028
## 4                       ROC-AUC winner Lasso (small)
## 5            ROC-AUC margin (1st - 2nd)        0.0229
## 6 Test-set prevalence (PR-AUC baseline)        0.0067
## 7             PR-AUC lift over baseline          6.5x
## 8                      Recommended model Lasso (small)
```

On the 30% test set, the two Lassos (small-category and big-category), the GAM, the multilevel logistic, and the RBF SVM are evaluated on the same observations using the same probability scale. PR-AUC is the primary criterion at the global level. For the rare-event task, however, the operating-point metrics in the previous subsection are the more honest read, because they describe what a curator would see when inspecting the top K% of predictions. The AUC table and the operating-point table together cover both the global ranking quality and the practical capture-precision trade-off.

The leading model on PR-AUC is the Lasso (small). ROC-AUC tells a parallel story, with the Lasso (small) again in front. The other models contribute different views of the same data: the coarse Lasso reports the L1 fit at the big-category resolution and quantifies the cost of collapsing the 248 small categories into 18 groups; the GAM exposes the non-linear shape of $f_1(\log(1 + \text{reviews}))$; the multilevel logistic adds partial pooling across the natural nesting of small categories within big-category groups; and the RBF SVM tests whether unstructured non-linear interaction in the numeric block adds anything beyond the additive view of the GAM.

Discussion and Limitations

The filter `price > 0 & stars > 0` removes about 11% of the raw catalog. The excluded rows are not missing at random: they correspond to free or promotional listings, items not yet rated, and entries with malformed price fields. To the extent that these mechanisms correlate with the outcome, the estimated effects describe the population of priced and rated products rather than the marketplace as a whole.

The 0.67% prevalence has two consequences. First, weighting strategies are unavoidable: without inverse-prevalence weights the loss is dominated by the negative class and the estimated probability surface collapses toward zero. Second, the spline basis in the GAM extrapolates aggressively in the upper tail of `log_reviews`, where positive observations are sparse. Wide confidence bands in that region are an honest signal of low information, not a defect of the model.

The category mix is non-uniform but is not dominated by any single group. Apparel-related categories together account for about 29% of the sample, and the 3 most populated small categories each represent under 2%. Lasso coefficients are estimated relative to the implicit reference defined by the model matrix construction, so their numerical interpretation is bound to that reference rather than to a single named baseline. The RBF SVM is trained on a stratified subsample, which limits the conclusions that can be drawn about its frontier; a different kernel choice or a richer feature design might recover signal that the present setup leaves on the table.

Conclusion and Future Work

Smoothing splines in the GAM recover the diminishing-returns pattern of review volume that a linear model cannot represent. Once that non-linearity is accounted for, price contributes a modest negative effect, and stars contribute a positive effect that, in absolute magnitude, is the largest among the continuous predictors. Reputation thresholds, review volume and rating, dominate price as drivers of Best Seller status in this sample, with the rating signal carrying more weight than review volume in the linear-additive fit.

3 findings about category structure are robust across the model suite. First, replacing the 248 small categories with the 18 big_category groups in the Lasso loses discriminative content: the small-category resolution carries information that partial pooling does not recover here. Second, the multilevel logistic underperforms the fine-grained Lasso on both PR-AUC and the operating-point metrics. With 0.67% prevalence and on the order of ten random levels per parent group, the random intercept shrinks toward each parent mean and washes out within-group heterogeneity that the unpenalized one-hot would have preserved. Third, the RBF SVM does not improve over the additive GAM in the numeric block, which suggests that non-linear interaction between `log_price`, `log_reviews`, and `stars` is small in this sample; the additive view of the GAM already captures most of the signal in the continuous predictors.

For the rare-event capture problem, the operating-point metrics are the natural tie-breakers, and they identify the Lasso (small) as the recommended model on the basis of its top-K precision and lift. The multilevel logistic remains the natural anchor for any extension that wants formally calibrated category-level effects under partial pooling.

Future work. A panel of weekly snapshots would permit time-series analysis of how visibility accrues and decays. Also, product titles are an unused source of signal. NLP features, such as sentiment scores, brand mentions, and category-aware embeddings, would complement the structured predictors used here and likely improve discrimination in the long tail of low-review products.